CHAPTER 1
INTRODUCTION

1.1 GENERAL

In today's digital era, the amount of sensitive data stored and shared over online platforms has increased rapidly. Organizations and individuals rely on file storage systems to manage confidential documents, business reports, intellectual property, and personal information. However, this growing dependency on cloud-based storage solutions has led to a parallel rise in data breaches, unauthorized access, and cyberattacks. Attackers constantly attempt to infiltrate storage systems through malicious uploads, compromised credentials, or internal manipulation of system logs to erase their traces.

Traditional file storage systems follow a centralized structure, where a single administrator or server fully controls stored data and access logs. While these platforms provide basic security features, they often fall short in ensuring data integrity and tracking malicious activities effectively. Logs can be altered or deleted by privileged insiders or advanced attackers, making forensic analysis and incident response extremely difficult. Moreover, standard antivirus methods may fail to detect emerging threats or hidden malicious payloads within uploaded files.

To address these challenges, there is a strong need for a secure, intelligent, and transparent file management solution. The proposed system introduces a Secure File Storage and Sharing Platform that integrates multiple security layers. It incorporates encryption to safeguard files during storage and transmission, while a Python-based threat detection engine evaluates uploaded files for potential risks before allowing them into the repository. Additionally, the system introduces blockchain technology for immutable audit logging. Every significant event such as user login, file upload, download, or blocked attack attempt is permanently recorded as a blockchain transaction, ensuring that no activity can be altered or hidden.

The system is designed using Java Spring Boot for the backend, ensuring efficient access control, communication, and server-side processing. Python microservices operate independently to analyze threat intelligence, and MySQL maintains secure metadata storage. A user-friendly interface allows both users and administrators to interact seamlessly with the system, while real-time threat alerts and monitoring features strengthen overall security governance.

1.1.1 Importance of Secure File storage and sharing with threat detection and blockchain - based audit

In the current era of digital transformation, the majority of business and personal information exists in electronic form and is continuously shared across networks and cloud platforms. This convenience, however, brings a parallel escalation in cybersecurity threats. Cybercriminals often exploit file-sharing systems to introduce malicious content, conduct data theft, or manipulate sensitive records. Even internal staff with high privileges may intentionally modify or delete audit logs to conceal unauthorized activities. The consequences of such breaches can be disastrous, including financial loss, identity theft, operational failure, legal complications, and irreversible damage to organizational reputation. Therefore, it has become increasingly essential to adopt advanced, multi-layered security practices in data storage infrastructures.

The importance of integrating secure storage, intelligent threat detection, and transparent audit mechanisms lies in ensuring complete data protection across all stages of file handling. Encrypting files during upload, storage, and retrieval ensures that even if an attacker gains access to the system, the content remains confidential and unusable. Role-based access control guarantees that only intended users are permitted to view, download, or share particular files, reducing insider threats. However, traditional security measures alone are no longer sufficient. Modern cyberattacks are becoming more sophisticated, often hidden within seemingly harmless files such as PDFs, images, or compressed folders. To counter these risks, incorporating a Python-powered threat detection engine becomes essential. By analyzing file metadata, structure, and behavior patterns, the system can identify and block harmful content before it enters the storage environment. This proactive defense approach not only prevents attacks but also reduces the spread of malware across connected systems.

Another crucial aspect of secure file management is maintaining trustworthy and tamper-proof audit logs. In centralized systems, logs are stored in standard databases where they can be modified, erased, or manipulated by privileged users or attackers who gain access. This creates significant challenges in forensic investigations, compliance audits, and accountability tracking. To solve this long-standing security flaw, blockchain technology offers a revolutionary solution. Blockchain maintains audit logs as decentralized, immutable records where no entry can be altered once written. Every action such as upload, access attempt, download, or security alert is recorded as a transparent transaction that is cryptographically secured. This assures that security incidents cannot be hidden and every user is held accountable for their actions.

1.1.2 Types of Attacks in File Storage and Sharing Systems

File storage and sharing systems are widely used across organizations to store sensitive data such as personal records, financial documents, confidential project files, and proprietary business information. As these systems evolve, cyberattackers continuously find new ways to compromise them. Attack scenarios include external threats from hackers as well as insider threats from authorized employees misusing access privileges. Without a multi-layered security strategy, attackers can cause significant damage including data theft, corruption, unauthorized disclosure, loss of confidentiality, operational shutdown, and manipulation of evidence. The following are major attack types that pose severe risks to centralized storage platforms:

One of the most common threats is malware-based file upload attacks, where harmful payloads such as ransomware, spyware, and trojans are embedded within documents or compressed files. When executed, they can spread through the network, encrypt data, or allow attackers to control the system remotely. Another major risk comes from unauthorized access attacks, where attackers bypass authentication mechanisms using stolen credentials, brute force techniques, or social engineering to access confidential files without detection. This leads to data leaks, breaches, and exploitation of sensitive information.

Insider attacks remain one of the most dangerous risks, as employees with high-level privileges may intentionally extract confidential data or delete logs that trace their actions. Since traditional systems store logs in editable databases, it becomes easy for insiders to manipulate or erase evidence of their misconduct. This results in loss of traceability and failed auditing processes. In addition, attackers exploit SQL injection vulnerabilities to access the system database directly, where they can manipulate user accounts, file metadata, or even gain full administrative privileges.

Another critical threat is Cross-Site Scripting (XSS), where attackers inject malicious scripts into the storage portal or uploaded metadata, allowing unauthorized execution inside the victim's browser. This may steal login sessions or trigger harmful automated operations. Large-scale DDoS attacks attempt to overwhelm the file storage server with massive traffic, causing downtime and denial of legitimate user access to stored files. Man-in-the-Middle (MitM) attacks also target unencrypted communication channels, manipulating or intercepting files during upload or download.

Attackers can also exploit file system vulnerabilities through Directory Traversal, enabling them to download system-level or confidential files outside their permissions. Log manipulation attacks target centralized logging systems where entries can be modified or deleted, making it impossible to track unauthorized actions. Finally, ransomware attacks aim to encrypt all stored data and demand payment for restoration, leading to business paralysis if backups and monitoring are insufficient.

1.1.3 Impact of Attacks

Attacks on file storage and sharing systems lead to severe consequences that affect individuals, organizations, and even national infrastructures. Since file storage systems hold confidential information including business documents, intellectual property, personal records, and strategic data, any compromise can result in catastrophic losses. The impacts extend beyond technical damage and influence financial stability, legal accountability, trustworthiness, and the continuity of operations. These impacts emphasize the urgency of implementing advanced protection mechanisms in modern data management platforms.

One of the most immediate impacts is data theft and unauthorized disclosure. Attackers may extract sensitive business files or personal information and misuse them for identity theft, corporate espionage, or competitive advantage. Such disclosure can permanently damage an organization's reputation and create long-term distrust among customers and partners. Another significant impact arises when malware enters storage systems through infected file uploads. These harmful programs may encrypt data through ransomware, corrupt files, or silently monitor user activities, leading to operational disruption and costly recovery efforts.

Data manipulation is another critical concern. Attackers or malicious insiders may alter stored information, causing inaccuracies, misinformation, and decision-making failures. In environments such as healthcare, finance, and government operations, manipulated files may even threaten lives or lead to policy and financial errors. Additionally, log tampering or deletion of access records can erase evidence of illegal activities, making forensic investigations and legal compliance extremely difficult. Without trustworthy audit trails, organizations cannot prove accountability or detect long-term intrusion.

Attacks can also result in service unavailability. Through Distributed Denial of Service (DDoS) attacks or storage corruption, legitimate users may lose access to their essential data. Downtime in business-critical environments causes revenue loss, customer dissatisfaction, and business failure. In some cases, attackers gain control over administrative privileges and lock users out of their own data, demanding ransom payments to restore access. Even after recovery, organizations often suffer lasting productivity reduction and system instability.

Financial costs of these attacks are significant as well. Organizations may incur high expenses in incident response, digital forensics, system restoration, legal settlements, and regulatory penalties. Data protection regulations such as GDPR, HIPAA, and IT Act impose strict liability on institutions that fail to secure sensitive data. Furthermore, once a breach becomes public, brand damage and customer churn intensify the financial impact over years.

The long-term consequences of cyberattacks also include psychological stress among affected users, loss of intellectual property, and strategic disadvantage in industries like technology and defense. These cascading effects demonstrate that threats to file storage systems are not simply technical issues but serious business and societal risks.

The severity of these impacts reinforces the necessity of a system like the proposed secure file storage and sharing platform. By integrating pre-upload threat detection and blockchain-based immutable audit logging, the proposed solution helps prevent attacks before they occur and ensures that every action remains traceable and accountable. Thus, organizations can safeguard critical data, maintain confidentiality and trust, and avoid long-lasting damage from security breaches.

1.1.4 Overview of File storage and Sharing with Threat detection

File storage and sharing platforms provide a centralized solution for storing, accessing, and distributing digital files in an efficient manner. These systems are widely adopted across organizations for collaborative operations and secure access to important information. Modern file storage architectures typically rely on cloud hosting, role-based permission management, encrypted content handling, and database-backed metadata tracking. Users are able to upload and download their files from any location, making these systems a vital component in today's digital infrastructure. Along with accessibility and convenience, they also carry the responsibility of ensuring strong data protection from cyber threats.

However, the growing sophistication of cyberattacks has revealed limitations in traditional storage systems. Attackers often utilize file upload channels to deliver malicious payloads such as worms, ransomware and trojans that remain hidden within seemingly harmless documents. Once stored, these harmful files can compromise the system, access sensitive databases or spread into other user devices. Threat actors may also perform unauthorized access, manipulate stored data, or erase system logs to hide their activities. These attack strategies demonstrate the need for proactive defense mechanisms rather than relying purely on manual inspection or basic antivirus scanning.

To counter these risks, modern secure file storage systems integrate detection techniques that analyze files before they are accepted into the system. Threat detection modules are typically implemented using rule-based scanning, signature matching, behavioral analysis, or machine learning classifiers. By extracting and evaluating file attributes such as structure, metadata, content signature, and activity patterns, the system can determine whether the file is safe or potentially harmful. If a threat is detected, the upload action is halted immediately, preventing the attacker from injecting malicious content into the repository.

In addition to proactive malware detection, modern security models also require transparency and accountability of user actions. Even with strict permissions, insider attacks remain a major concern. To address this vulnerability, immutable logging techniques such as blockchain audit trails are increasingly utilized. Blockchain ensures that all critical events like upload, access attempts, and detection alerts are permanently recorded in a tamper-proof ledger. No administrator or attacker can alter these records, thus guaranteeing forensic traceability of all user activities.

By combining secure storage, threat detection intelligence, and immutable accountability, the latest advancements in file storage systems offer holistic protection against both external and internal cyber risks. These integrated security layers create a reliable and trustworthy storage environment suitable for sensitive applications such as healthcare, finance, defense, research, and legal services. Therefore, the importance of including detection techniques in modern file sharing systems is indispensable for protecting data integrity, preventing cyberattacks, and ensuring a resilient and compliant information system.

1.1.5 Role of Machine Learning

Machine Learning (ML) plays a crucial role in enhancing the security of file storage and sharing platforms by enabling intelligent detection of malicious activities and harmful content. Traditional security mechanisms mainly rely on signature-based detection, where known threat signatures are matched to uploaded files. However, attackers continuously create new malware variants that escape such conventional scanning techniques. Machine Learning brings the capability to automatically learn patterns, behaviors, and hidden relationships in data that indicate potential cyber threats, enabling proactive defense against evolving attacks.

In the proposed system, Machine Learning is primarily used to analyze files before storage to determine whether they may contain harmful elements such as ransomware payloads, embedded scripts, or obfuscated malicious code. Various features of the file such as size, type, structure, header data, metadata, compression behavior, or hash-based characteristics can be extracted and fed into ML models. The model identifies patterns consistent with past harmful files, even when the exact malware signature is unknown. This helps in identifying zero-day threats that antivirus tools may miss.

Machine Learning also helps in behavioral threat detection, where irregular user actions such as repeated failed login attempts, unusual access times, unauthorized file downloads, or sudden spikes in upload activity can be analyzed to detect insider threats or compromised accounts. Instead of manual monitoring, ML algorithms can automatically classify these patterns as normal or suspicious and trigger alerts in real time.

Additionally, ML enables continuous improvement of the threat detection module. As the system processes more files and threat cases, new data can be used to retrain and refine the model, increasing accuracy over time. This capability to learn from experience distinguishes Machine Learning from static rule-based security systems. The integration of ML therefore makes your platform adaptive, scalable, and future-ready, capable of defending against both known and unknown cybersecurity threats.

Another benefit is the reduction of false positives, ensuring legitimate files are not mistakenly blocked. Machine Learning models evaluate thousands of subtle features during analysis, making decision-making more precise and reliable. Furthermore, ML-driven insights contribute to forensic analysis, risk scoring, and prioritizing alerts, helping administrators respond faster to threats.

In summary, Machine Learning enhances the intelligence layer of your secure file storage system by enabling predictive analysis, early threat detection, anomaly recognition and automated decision-making. This strengthens overall system resilience, maintaining high availability and trust while preventing unauthorized or malicious content from being stored, shared, or executed within the organization.

1.1.6 Challenges in Detection

Detecting malicious content during file upload is a complex and continuously evolving task. Attackers are always developing new tactics to conceal harmful payloads, making threat identification a major cybersecurity challenge. Even with modern Machine Learning and behavior-based detection strategies, numerous limitations and uncertainties must be addressed to ensure reliable security. These challenges influence the accuracy, performance, and overall trustworthiness of the detection module in secure file storage environments.

One of the major challenges lies in detecting zero-day malware, which refers to newly developed threats that have never been encountered before. Since these malicious files do not match any known patterns or signatures, identifying them requires advanced models capable of studying deeper behavioral and structural features. However, training such models requires extensive datasets, and the lack of labeled samples for new viruses slows down accurate analysis. Another challenge arises from obfuscated malware, where attackers deliberately disguise executable code inside common file types such as PDFs, Word documents, or compressed archives. These files appear safe but activate malicious behavior once accessed by the user.

High false-positive rates are another major issue. Overly sensitive detection systems may block legitimate files that contain unusual metadata or structures used for valid purposes. This affects user experience, disrupts workflows, and may cause users to distrust the security system. On the other hand, false negatives allow dangerous files to pass through undetected, resulting in significant security compromise. Balancing accurate detection and usability remains one of the hardest problems in cybersecurity.

The computational cost of real-time scanning also poses a significant challenge. Large files, media content, or high-volume uploads require more processing time, slowing down the system's responsiveness. Deploying Machine Learning models in a live server environment must be optimized to avoid performance bottlenecks and latency, especially when many users interact simultaneously.

Another serious challenge is the detection of insider threats and behavioral anomalies. Users with legitimate permissions may engage in harmful actions such as unauthorized sharing, over-downloading sensitive data, or altering security controls. Recognizing malicious intent behind seemingly valid user activity requires advanced behavioral analytics, which depend on continuous monitoring and well-defined risk scores.

Attackers also attempt evasion techniques, such as encrypting malicious code to bypass detection engines, altering file structure dynamically, or using AI-generated malware patterns. This creates a constant race between attackers and defenders, requiring continuous model retraining and updating of detection strategies. Additionally, integrating ML-based detection introduces challenges related to model drift, where accuracy declines over time if the system does not learn from new threat behaviors.

Privacy concerns also influence detection capabilities. Deep inspection of content may compromise sensitive information stored within the file. Therefore, detection models must ensure the analysis does not violate data confidentiality and must handle encrypted files responsibly.

In summary, threat detection faces several challenges including hidden malware, evolving attack patterns, performance limitations, false alarms, and insider risks. Overcoming these challenges requires a combination of advanced detection models, continuous learning, efficient processing techniques, and strong security governance. The proposed system helps mitigate many of these issues by analyzing multiple features, using risk-based decisions, and reinforcing detection with blockchain-based accountability for actions.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of File Storage & Sharing with threat detection

Secure file storage and sharing has been widely studied in the context of cloud computing, access control, and data confidentiality. Early solutions mainly relied on centralized servers with encryption and access policies enforced at the application or database layer. Recent research has moved towards decentralization and trust minimization using blockchain technology. For example, Putz et al. proposed a secure and auditable logging infrastructure based on a permissioned blockchain that ensures log integrity and non-repudiation without depending on a single trusted provider. Their work shows that blockchain can serve as a distributed auditing layer where log entries can be independently verified by different parties.

On the threat detection side, especially malware and malicious file detection, Machine Learning has become a dominant technique. Singh et al. presented a comprehensive survey on machine-learning-based malware detection, emphasizing that ML models can identify complex and previously unseen malware patterns more effectively than traditional signature-based methods.

2.1.1 Techniques used

To build a robust and trustworthy file storage & sharing system, a combination of complementary techniques is required. Below are the core techniques that form the backbone of such a system, and how they play together to ensure security, detection, and accountability.

Behavioral / I/O pattern + ML detection: Several works show it's effective to detect ransomware or malicious files by analyzing access patterns or file metadata (entropy, read/write behavior) rather than pure signature matching. Good support for implementing a Python-based detection engine that works before storage.

Encryption + deduplication + secret sharing for efficiency + security: Storage efficiency (deduplication) along with strong confidentiality can be balanced using smart encryption/key-management strategies published in research, which helps when storing large volumes of data.

2.1.2 Machine Learning Approaches

Machine learning has become one of the most prominent techniques in recent research for detecting malicious files and security threats in storage and sharing platforms. Many studies adopt supervised learning models such as Random Forest, Decision Tree, Support Vector Machine, and Neural Networks to classify files as benign or harmful based on static features extracted from file metadata, structure, and byte-level characteristics. Other works focus on behavior-based detection, where models analyze system calls, storage-level access patterns, and user activities to identify ransomware or unauthorized behavior before damage occurs.

To improve resilience against zero-day threats and obfuscated malware, researchers have explored hybrid and ensemble approaches, combining static and dynamic analysis methods with machine learning to enhance detection accuracy and reduce false alarm rates. Recent advancements also include the use of deep learning and automated feature extraction to detect complex malicious patterns that traditional tools often miss. Overall, the literature strongly supports machine learning as a powerful and adaptive defense mechanism that significantly enhances threat detection in secure file storage environments.

2.2 Supervised Learning Approaches

Supervised learning is one of the most widely used machine learning approaches for detecting malicious files and abnormal activities in secure file storage and sharing systems. In supervised learning, the model is trained using a labeled dataset that contains examples of both benign and malicious files. The learning algorithm studies the patterns, statistical characteristics, and distinguishing features of each category so that it can classify newly uploaded files accurately during real-time operations. This makes supervised learning particularly useful for preventing the upload and execution of harmful files in a cloud or enterprise storage setting.

Supervised learning techniques rely on a feature extraction process where relevant static or behavioral file traits are gathered from input samples. These features may include file size, entropy level, headers, embedded scripts, opcode patterns, metadata, frequency of system calls, access patterns, or network behavior during execution. Once the dataset is prepared, the model learns decision boundaries that differentiate legitimate files from malicious threats. After training, the classifier can automatically evaluate any uploaded file and provide a prediction such as “safe,” “suspicious,” or “malicious,” along with a confidence score that helps in decision making.

Among supervised learning algorithms, Decision Trees and Random Forests are commonly used due to their interpretability and high performance with heterogeneous file features. Decision Trees learn a sequence of rules that lead to classification, making it easier for analysts to understand model behavior. Random Forest, an ensemble of multiple trees, improves accuracy and reduces the risk of overfitting, making it ideal for malware detection scenarios where malicious patterns may vary widely. Support Vector Machines (SVM) are also frequently used because they perform well with high-dimensional feature vectors and can create strong separation between legitimate and malicious classes with kernel-based boundary transformations.

Several studies apply Artificial Neural Networks (ANNs) to automatically detect patterns that are not human-interpretable. ANNs are capable of learning complex non-linear relationships in data and are effective when detecting obfuscated or polymorphic malware. More advanced deep learning models utilize layers of automated feature extraction that analyze raw byte sequences or file binaries, reducing reliance on handcrafted feature engineering. These advanced models offer improved detection rates but may require higher computational resources, which must be considered when deploying them in real-time systems.

Supervised learning models can be continuously improved by retraining them with new malware samples and recently observed attack behaviors. However, they depend heavily on the quality and diversity of the labeled datasets. If the dataset does not include enough examples of emerging malware families or sophisticated insider attacks, the model may fail to generalize effectively and produce false negatives. To overcome such issues, researchers often combine supervised learning with heuristic analysis, dynamic monitoring, and ensemble strategies to strengthen detection reliability.

Overall, supervised learning provides an intelligent and automated approach to detecting harmful uploads in file storage environments. It enables early prevention, reduces human effort in identifying malicious activity, and significantly improves security posture. When integrated with blockchain-based logging and secure encryption mechanisms, supervised learning forms an essential defensive layer for modern secure file storage and sharing platforms.

2.3 Unsupervised Learning Approaches

Unsupervised learning plays a vital role in enhancing security within modern file storage and sharing systems by enabling the detection of unknown, hidden, and evolving threats without requiring labeled datasets. Unlike supervised learning, which depends on clear distinctions between benign and malicious files during training, unsupervised approaches rely on identifying unusual patterns, deviations from normal behavior, and structural anomalies in data. This is particularly important in cybersecurity contexts where attackers continuously modify malware signatures to evade traditional scanners, or when new malware variants emerge without historical reference samples. Because file storage platforms regularly handle a wide variety of content and user activity patterns, unsupervised learning becomes essential for real-time anomaly detection and proactive threat prevention.

One commonly used unsupervised technique is clustering, where files or user behaviors are grouped into clusters based on similarity. Algorithms such as k-means, DBSCAN, and Gaussian Mixture Models classify normal user actions or benign file types into dense groups, while abnormal behavior falls into sparse, distant clusters and is flagged as suspicious. This enables automatic identification of unexpected file uploads, excessive download activities, or unauthorized access patterns. Another effective method is density-based anomaly detection, including Local Outlier Factor (LOF), which detects threats by locating data points in regions of low distribution density. Such approaches are highly effective when attacks represent only a small portion of overall system activity.

Isolation-based detection techniques, such as Isolation Forest, are widely used because of their efficiency in large-scale environments. These models isolate anomalies faster since unusual instances require fewer random partitions to separate them from normal data. These methods are well-suited for cloud and enterprise storage systems that process high volumes of files with varying characteristics. Additionally, One-Class Classification models, like One-Class SVM, learn only from legitimate usage patterns and mark anything outside normal boundaries as a potential threat. This approach is valuable in settings where malicious data samples are unavailable or unpredictable.

Advanced unsupervised learning solutions apply Autoencoders, which are neural networks trained to reconstruct normal behavior with minimal error. When a new file or activity deviates significantly from the learned structure, the reconstruction error becomes high, indicating a

probable anomaly. Autoencoders have shown strong results in detecting stealthy malware and insider misuse by monitoring file access sequences, login behavior, and metadata changes. Similarly, dimensionality reduction techniques such as Principal Component Analysis (PCA) identify previously unseen attack vectors by highlighting abnormalities hidden in complex, high-dimensional feature spaces.

Despite their advantages, unsupervised models can be challenging to optimize because they must distinguish between legitimate but rare behavior and genuinely malicious anomalies. Poorly tuned models may generate false positives or miss subtle threats. However, when combined with supervised detection, blockchain-based logging, and encryption strategies, unsupervised learning significantly strengthens the security of file storage systems by offering continuous learning and adaptive defense mechanisms.

In summary, unsupervised learning approaches are highly effective in identifying zero-day attacks, anomalous file uploads, and suspicious user activities without requiring prior knowledge of attacker strategies. Their self-learning ability, scalability, and resilience to evolving threats make them an essential security component in modern secure file storage and sharing environments.

CHAPTER 3

PROPOSED METHODOLOGIES

3.1 System Design

The proposed system is architected as a secure, intelligent, and accountable file storage and sharing platform that integrates multi-layered security controls, real-time threat detection, and immutable audit logging. The system adopts a modular and layered design, ensuring that each functional component is isolated yet seamlessly cooperates with others to provide strong confidentiality, integrity, and transparency during file operations. The core architectural layers include the Presentation Layer, Application/Service Layer, Security Analytics Layer, Audit Logging Layer, and Data Management Layer. These layers operate together to ensure that every file uploaded to the system is thoroughly validated, securely processed, and permanently monitored.

The Presentation Layer consists of a web-based interface where users and administrators interact with the system through secure authentication. Users can upload, download, and manage files, while administrators monitor audit logs, suspicious activities, and permission controls through a dedicated dashboard. All communication from the UI to the backend is tunneled through HTTPS, preventing eavesdropping and data interception attacks.

At the core lies the Application Layer, developed using Java Spring Boot, which implements all business logic. This layer performs user authentication and authorization through token validation and role-based access control, ensuring that only permitted users can perform sensitive actions. The File Management module handles all file operations, including secure upload, controlled access, and encrypted storage. When a file is submitted, it is first examined before storage, and no data is directly placed into the repository without validation. This Application Layer also interacts with external modules for security enhancement, making the architecture scalable and updatable over time.

The Security Analytics Layer is implemented as a separate Python-based microservice responsible for threat detection. When a file is uploaded, the Java backend extracts relevant features such as file size, metadata, structure, and content signatures, forwarding them to the Python service for intelligent analysis. Machine learning and heuristic techniques evaluate the

risk level of the file, determining whether it is malicious, suspicious, or safe. Based on the threat detection outcome, the backend either allows or denies the upload request. This real-time scanning prevents ransomware, malware-embedded content, and harmful scripts from entering the system.

To reinforce trust and accountability, the Audit Logging Layer integrates blockchain technology to store immutable logs of security-critical events such as login attempts, uploads, downloads, data access violations, and flagged threats. Once recorded, these logs cannot be deleted or modified, even by system administrators, thereby preventing insider manipulation and ensuring verifiable transparency. The blockchain ledger acts as a tamper-proof evidence repository for forensic and compliance needs.

Finally, the Data Management Layer consists of two secure storage mechanisms: an encrypted file storage system that protects file contents from unauthorized access and a structured relational database (MySQL) that manages user credentials, file metadata, access permissions, and threat history. All sensitive user data, including passwords, is securely hashed before storage, ensuring compliance with privacy and data protection principles. File encryption keys are securely managed, ensuring that even if the storage environment is compromised, no meaningful data can be extracted.

```
+---------+        +-------------+        +-----------------+        +------------------+
|  User   | -----> |  Web UI /   | -----> |  Java Backend   | -----> | Python Threat    | ----
| (Client)|        |  Frontend   |        | (Spring Boot)   |        | Detection Service|
+---------+        +-------------+        +-----------------+        +------------------+
                                             |      |    \
                                             |      |     \
                                             |      |      \
                                             v      v       v
                                        +--------+  +-----------+
                                        | MySQL  |  | Encrypted |
                                        |  DB    |  | File Store|
                                        +--------+  +-----------+
```

Figure 1 Data Flow Diagram of the File Storage & Sharing System with Threat Detection and Blockchain -Based Audit .

3.1.1 Architecture of the Detection Tool

The Authentication and Access Control Module is responsible for ensuring that only legitimate users with valid credentials can enter the system. It securely manages login, registration, and session validation using strong hashing mechanisms and token-based authentication. This module also enforces role-based access control, meaning general users and administrators have different privileges within the system. Each file operation, whether upload, download, or sharing, is checked against the user's role and permission level to ensure compliance with access policies. Any unauthorized attempt is blocked and recorded for audit and security monitoring.

The File Management Module handles all processes related to secure file uploading, storage, downloading, and sharing. When a user initiates an upload, the file is first temporarily processed before being approved for permanent storage. This prevents malicious content from bypassing security layers. If the file is deemed safe after inspection, it is encrypted and saved in the secure storage repository, while metadata about the file such as owner, timestamps, and access permissions is stored in the database. During download requests, the module verifies access permissions, decrypts the file, and delivers it securely to the authorized user. This module ensures confidentiality of stored data and prevents unauthorized disclosure or tampering of stored files.

The Threat Detection Module, implemented as a separate Python-based microservice, is the analytical intelligence layer of the system. It examines uploaded files using a combination of machine learning and heuristic techniques to identify potential malware, ransomware, or abnormal structures. Instead of directly trusting files submitted by users, this module classifies them based on computed risk scores and security patterns. If a file exhibits suspicious attributes, the module alerts the backend to deny the upload and notify the administrator. This proactive detection mechanism greatly reduces the risk of harmful files being stored or executed within the system.

The Blockchain Audit Logging Module reinforces system accountability and log security by permanently recording all critical events, such as upload attempts, access changes, detection alerts, file downloads, and failed authentication activities. Unlike traditional centralized logging

methods where administrators can alter or delete records, blockchain technology makes logs immutable and tamper-proof.

The Database Management Module provides structured storage for all system-related metadata. It stores user accounts, hashed passwords, file information, permission settings, and threat detection records. The relational database management approach maintains data consistency and integrity while supporting fast querying for dashboard display, file lookups, and authorization checks. The module acts as an information backbone that keeps track of all linkages between users, files, permissions, and logged events.

Finally, the Web User Interface Module serves as the interaction layer between the system and its users. It provides a convenient and user-friendly dashboard through which users can upload and manage files, and administrators can monitor system health, analyze alerts, and verify blockchain records. Communication from the UI to the backend is performed through secure HTTPS calls, ensuring files and credentials remain protected during transfer. The interface simplifies complex security operations behind a clear and accessible layout, making the system practical for real-world organizational usage.

Together, these six modules work cohesively to establish a secure storage ecosystem where data is protected through encryption, intelligent threat identification, and tamper-proof logging. This modular design provides flexibility, scalability, and strong resistance against cyberattacks, fulfilling the core objectives of the proposed secure file storage and sharing system.

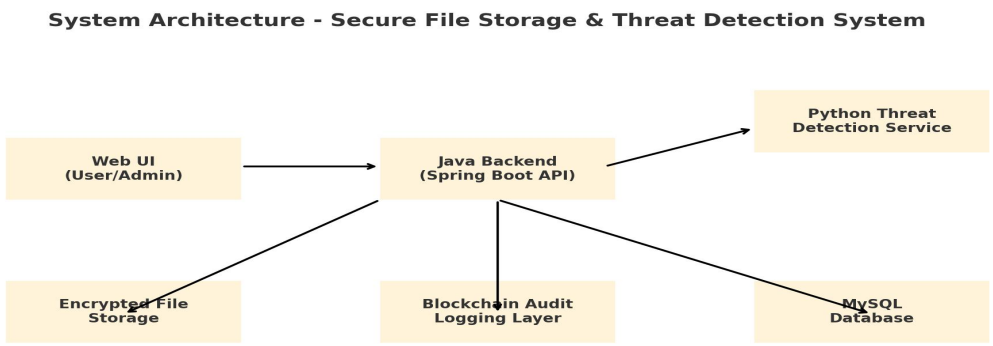

Figure 2 Architecture of the File Storage and Sharing with Threat Detection System

3.2 Feature extraction and selection

Feature extraction is a crucial step in the threat detection process of the proposed secure file storage and sharing system. Before any machine learning model can classify an uploaded file as safe or malicious, the raw file data and associated user activity must be converted into meaningful numerical values called features. These features capture important characteristics of the file and its context, allowing the Python-based detection engine to learn hidden patterns that differentiate normal uploads from potentially harmful ones. In the proposed system, feature extraction operates at two main levels: file-level features and behavioral/contextual features.

At the file level, the system first analyzes basic properties of each uploaded file. This includes the file size, file extension, MIME type, and upload time. These simple attributes already provide useful signals; for example, an executable file uploaded in a system where only documents are expected may be suspicious. The system also computes file hash values (such as SHA-256) to uniquely identify files and check for known malicious signatures or repeated uploads. Additionally, the detection module calculates entropy-based features which indicate the randomness of the file content. Very high entropy often appears in compressed or encrypted malicious payloads, whereas many benign office documents show more regular structure. By combining size, type, hash, and entropy, the model gains a robust static view of the file.

Beyond basic metadata, the system can extract content-structure features from the file. For document formats (PDF, DOCX, etc.), it examines the presence of embedded objects, scripts, macros, or unusual headers. For example, a document containing an abnormally high number of embedded scripts or suspicious macro instructions can be given a higher risk score. For binary files, the feature extractor may consider byte distribution statistics, n-gram frequency patterns, or section characteristics (e.g., presence of suspicious import functions in PE headers). Although the implementation can be simplified for academic scope, the design is flexible enough to support richer low-level features in future enhancements.

In addition to static file characteristics, the proposed system makes use of behavioral and contextual features that describe how the file is being used within the platform. Each upload is associated with the uploader's user ID, role (user/admin), source IP address, and geolocation or network segment where applicable. Features such as the number of files uploaded by the same

user within a short time window, the distribution of file types uploaded by that user, and the history of failed login attempts provide valuable behavior context. For example, if a normal user suddenly uploads a large number of encrypted archives from a new IP address after several failed login attempts, this pattern may indicate account compromise or malicious intent even if each individual file looks normal.

The system also records time-based features, such as whether the upload occurred during regular working hours or at unusual times, and the frequency of download requests for recently uploaded files. These temporal patterns help the model recognize anomalies like sudden spikes in activity, massive downloads of sensitive content, or uploads happening only at midnight from unknown networks. All of these behavioral and contextual features are stored in the MySQL database and forwarded as structured inputs to the Python detection service when required.

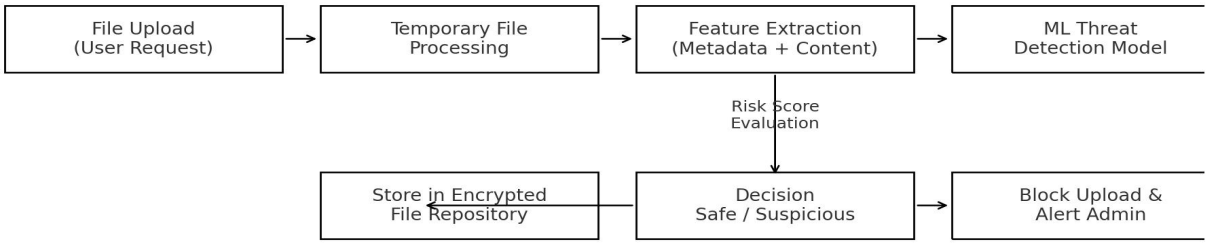

Figure 3 Feature extraction for File Storage & Sharing with Threat Detection and Blockchain - Based Audit

Once extracted, all features are normalized and encoded into a consistent numeric format suitable for machine learning. Categorical values like file type or user role are converted into encoded representations, while continuous attributes such as size, entropy, and upload count may be scaled to a common range. The resulting feature vector represents each upload event in a compact mathematical form. The supervised or unsupervised ML model then uses these feature vectors to learn what constitutes normal behavior and what should be classified as suspicious.

3.2.1 Threat Detection Features

Feature extraction is the most critical phase in the threat detection workflow of the proposed secure file storage and sharing system. Before the Machine Learning model can classify uploaded files as “safe” or “malicious,” the file must be transformed from raw binary content into

a structured numerical format that the model can interpret and evaluate. This process enables the system to analyze unique patterns, behavioral indicators, and structural characteristics that are often invisible to basic antivirus scanners.

To begin the analysis, the system first derives static features directly from the file's metadata. These include file size, file extension, MIME type, checksum values such as SHA-256, and the number of embedded resources. These simple attributes often reveal potential anomalies. For example, an .exe or .bat program uploaded under the disguise of a .pdf file format can raise an early alert. The SHA-256 hash can also serve as a signature to detect known malware that has previously been uploaded or blacklisted in external threat databases.

In addition to metadata, the system extracts content-based features, which are strong indicators of malicious behavior. One such feature is file entropy, which measures randomness within the file content. A very high entropy score may indicate hidden encrypted payloads or obfuscated malware, whereas low entropy is more typical of normal office documents and images. The system can also evaluate byte distribution patterns, header signatures, and n-gram character frequencies, which help the detection model identify encrypted droppers, macro-based exploits, or embedded scripts concealed inside document formats.

To strengthen detection accuracy, the system complements static analysis with contextual and behavioral features, focusing not only on what the file contains, but how and when it is being used. The feature extractor tracks the uploading user's identity, permission level, and prior history of flagged actions. Abnormal behaviors such as repeated uploads of encrypted compressed archives, access from unusual IP addresses, or sudden bursts of file-submission activity contribute additional features to the detection model. These indicators are important for detecting insider threats or credential compromise attacks, which may occur even when files appear harmless in isolation.

By combining metadata inspection, content structure evaluation, entropy and pattern analysis, along with user-behavior profiling, the feature extraction process forms an intelligent barrier that prevents advanced threats from entering storage. It acts not only as a static scanner but also as a dynamic risk assessor that adapts to evolving malware strategies. Without this feature extraction layer, even the strongest ML model would struggle to differentiate a concealed cyberattack from a legitimate file upload.

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 DEVELOPMENT ENVIRONMENT

The development environment for the proposed Secure File Storage and Threat Detection System is designed to support modular implementation, efficient communication between components, robust security mechanisms, and seamless integration of machine learning features. The project is developed using a combination of Java, Python, and modern web technologies, with strong support tools provided by Integrated Development Environments (IDEs), version control, and dependency management frameworks.

The primary backend of the system is implemented using Java Spring Boot, which provides a structured and scalable framework for developing security-focused web applications. Spring Boot enables rapid API creation, environment configuration, authentication handling, and secure communication workflows. To facilitate productive backend development, IntelliJ IDEA is used as the main IDE, offering advanced code navigation, debugging support, and built-in dependency management using Maven. IntelliJ's integration with Spring Boot significantly accelerates project setup, deployment, and server testing.

For the machine learning and file analysis component, the system uses Python, due to its strong ecosystem of libraries such as NumPy, Pandas, Sci-kit Learn, and TensorFlow, which simplify threat detection model implementation. The Python microservice runs independently using Flask or FastAPI, enabling efficient communication with the Java backend via REST APIs. Development and testing of the machine learning models are supported using tools like Jupyter Notebook or PyCharm Community Edition.

The interface for user and administrator interactions is built using modern web technologies that are compatible with Spring Boot's RESTful architecture, supporting secure authentication, encrypted data exchange, and responsive user experience. The frontend runs in a browser environment where HTTPS protocols ensure safe data transmission between client and server.

For data storage, the environment uses MySQL Database Server, configured for secure access and optimized query performance. The file storage is handled in a secured encrypted directory structure, ensuring confidentiality of uploaded content. Additionally, a Blockchain logging mechanism is incorporated through a lightweight private blockchain network or local node configuration to store tamper-proof audit trails.

The entire project follows a modular microservice design, allowing each major component to be developed, tested, and upgraded independently. Development, version control, and team collaboration are managed using GitHub, ensuring proper code tracking and controlled releases. The environment is deployed on systems running Windows 10 or above, with required installations such as Java Development Kit (JDK 17 or later), Python 3.9+, Apache Tomcat (optional for server hosting), and MySQL Workbench for database administration.

Overall, the development environment is designed to maximize security, flexibility, and scalability. The combination of Java Spring Boot for the backend, Python for intelligent threat analysis, MySQL for secure data management, and blockchain for audit integrity results in a powerful setup that supports both current functionality and future enhancement possibilities. This environment ensures efficient development cycles and provides a stable foundation for implementing advanced secure file storage solutions.

4.2 Java

Java serves as the primary backend programming language for the proposed secure file storage and threat detection system. It is chosen due to its platform independence, high performance, and strong security features, which are crucial for handling sensitive data and file operations. Java provides a stable runtime environment through the Java Virtual Machine (JVM), ensuring reliable execution across different deployment platforms. Its rich standard libraries and well-established ecosystem support secure encryption, authentication, database interaction, and HTTP communication. Java's multi-threading capabilities help manage simultaneous uploads and downloads efficiently, enabling a seamless user experience even under high workload conditions. Additionally, Java integrates seamlessly with Spring Boot, allowing the system to scale and adapt as more security features or storage services are added in the future.

4.3 Spring Boot

Spring Boot is used as the core backend framework to build secure and scalable RESTful services for the system. It provides auto-configuration, dependency management, and integrated security features that streamline development of enterprise-level applications. Spring Boot's layered architecture allows separation of responsibilities such as controllers, services, repositories, and security modules, ensuring clean and maintainable code. It also includes Spring Security, which strengthens authentication and access control through role-based permission enforcement, JWT token validation, and encrypted password handling. With built-in support for database connectivity, custom middleware, and cross-service communication, Spring Boot acts as the command center that coordinates file processing, threat detection requests, blockchain logging, and database operations efficiently.

4.4 Python

Python powers the intelligence portion of the system by hosting the machine learning-based file threat detection engine. Python is selected due to its extensive support for data science and cybersecurity libraries that make content analysis and anomaly detection highly effective. The Python module operates as an independent microservice, exposing APIs that accept file features from the Java backend and return a risk classification based on trained ML models. This modular approach ensures that updates to detection logic can be made without affecting the rest of the system. Python's flexibility and rapid development capabilities enable quicker experimentation with threat detection algorithms, making it ideal for continuous learning against evolving cyberattacks.

4.5 Scikit-Learn

Scikit-Learn is the backbone of the machine learning component used for file threat classification in the proposed system. It provides a wide range of supervised and unsupervised algorithms such as Random Forest, Support Vector Machine (SVM), Decision Trees, and Isolation Forest, allowing the detection engine to analyze file features and distinguish malicious uploads from legitimate ones. The library simplifies model training, testing, validation, hyperparameter tuning, and performance evaluation through tools like confusion matrices and accuracy metrics. Scikit-Learn's efficient implementations ensure fast inference times, making it suitable for real-time scanning environments .

4.6 Pandas

Pandas is used in the threat detection module for preprocessing and transforming extracted file features into structured datasets required for model input. Because machine learning models require clean and organized numerical data, Pandas plays a vital role in handling operations such as missing value treatment, feature normalization, label encoding, statistical grouping, and dataset merging. During the training phase, it allows analysts to combine metadata features, entropy scores, and user behavior attributes into unified data tables. Pandas DataFrames provide a secure and convenient structure to store detection logs and feature history for long-term analytics. This ensures the ML system has consistent feature patterns to learn from, while reducing noise and improving classification reliability.

4.7 Numpy

NumPy supports the mathematical computations required during feature extraction and machine learning evaluation. Many features used for threat detection, such as entropy values, byte-frequency distributions, and statistical variations in file structure, require fast numerical processing. NumPy arrays provide an optimized data format to represent feature vectors, allowing ML models to perform vectorized computations that are significantly faster than standard Python operations. The library is also essential during model training, where matrix operations, probability calculations, and data transformations must be handled efficiently. Its seamless integration with Pandas and Scikit-Learn enhances the overall performance and accuracy of the detection pipeline.

4.8 MySQL

MySQL plays a central role in the data persistence layer of the proposed secure file storage and sharing system. It is used to store structured information such as user profiles, hashed authentication credentials, role-based permissions, file metadata, and threat detection outcomes. The relational design supports strong normalization, ensuring consistency and integrity of stored data. MySQL provides reliable ACID-compliant transactions, which are crucial for maintaining accurate logs of file uploads, downloads, and access control decisions. Using tables and foreign key relationships allows seamless mapping of which user owns which file, who is granted access, and which security alerts are associated with a specific event. The database also holds cross-reference identifiers linked to blockchain transaction hashes, enabling verification of log integrity. Integration with Spring Data JPA and Hibernate simplifies secure data access and prevents SQL

injection attacks through parameterized queries. MySQL Workbench further supports database inspection, schema design, and query debugging. Overall, MySQL serves as the trusted storage foundation that ensures data organization, security governance, and smooth system operations.

4.9 Blockchain layer

The blockchain layer in the proposed system is responsible for ensuring the integrity and immutability of audit logs generated during critical user actions such as file uploads, downloads, failed authentications, and suspicious activity detections. Instead of relying solely on a traditional logging system where administrators or attackers could manipulate or delete records, blockchain guarantees tamper-resistant storage of event logs. Each important action is recorded as a transaction and linked using cryptographic hashing, forming a chain of proof that cannot be modified without breaking the entire chain structure. This ensures transparency, non-repudiation, and provides strong forensic evidence during security investigations. The blockchain operates as a lightweight private chain, eliminating public network delays while maintaining decentralization within your system environment. The Java backend interacts with blockchain nodes using secure API calls, generating transaction hashes stored alongside metadata in MySQL for cross-verification. By integrating blockchain for auditing, the system strengthens trust and accountability, protecting organizations from insider threats or unnoticed log manipulation.

4.10 React.js

React.js is used as the front-end framework in the proposed secure file storage and threat detection system to provide users with a dynamic, responsive, and intuitive interface. It enables the creation of reusable UI components, which helps maintain consistency across pages such as login, file upload, file dashboard, and admin monitoring screens. React's virtual DOM efficiently updates only the parts of the webpage that change, improving overall performance and ensuring a smooth experience even during frequent operations like file listing and refreshes. The architecture supports clean separation of UI components from backend logic, interacting with the Java Spring Boot APIs through secure HTTPS-based REST calls. React also integrates well with state management tools like Redux or Context API, allowing effective handling of user authentication states, session tokens, and alert notifications. Its strong community ecosystem, modern design patterns, and cross-browser compatibility make React.js an ideal choice for delivering a professional and user-friendly interface that supports real-time security status feedback and seamless navigation throughout the system.

4.11 IntelliJ IDEA

IntelliJ IDEA is used as the primary Integrated Development Environment (IDE) for developing and maintaining the Java Spring Boot backend. It provides intelligent auto-completion, real-time code inspection, refactoring tools, and debugging capabilities that significantly improve development productivity and reduce errors. IntelliJ's seamless integration with Maven, Git, Spring Boot Developer Tools, and REST API testing plugins allows efficient building, testing, and deployment of backend modules. The IDE also features built-in security code checks, helping developers detect vulnerabilities and misconfigurations early in development. With support for multi-module projects and microservice environments, IntelliJ IDEA ensures a smooth workflow for building secure and scalable enterprise applications.

4.12 HTTPS + Encryption Standards

To safeguard data confidentiality and secure communication, the system uses HTTPS enforced with SSL/TLS encryption. This ensures that sensitive content such as login credentials, file data, and threat detection results are protected while traveling across the network. Additionally, uploaded files are stored in encrypted form using strong symmetric encryption algorithms such as AES-256, preventing unauthorized access even if the storage environment is breached. Passwords and key identifiers are hashed using secure hashing algorithms like BCrypt or SHA-256 before being stored in the database.

4.13 GITHUB

GitHub is utilized as the version control and remote code repository platform for the entire project. It enables safe tracking of source code changes, maintaining commit history, and supporting coordinated team development. GitHub ensures that every update to backend, frontend, ML service, and database scripts is properly recorded and recoverable. Developers can work on separate branches for new features, perform code reviews, and merge updates without risking stability. It also facilitates CI/CD workflows and acts as secure off-site backup for the codebase, ensuring no loss of progress during system failures. Overall, GitHub supports quality control, team efficiency, and secure software lifecycle management.

CHAPTER 5

MODULES

5.1 IMPORTING REQUIRED LIBRARIES

The proposed secure file storage and threat detection system relies on a rich set of imported libraries across its backend, machine learning, blockchain, and frontend components to provide a secure, responsive, and intelligent operational environment. At the backend layer, Java Spring Boot imports essential libraries such as Spring Web for building RESTful APIs that manage file upload and download requests, and Spring Security, which enforces authentication, authorization, and encrypted credential handling. These libraries help implement strict role-based access control, preventing unauthorized operations on sensitive files. Additionally, Spring Data JPA is used for seamless integration with the MySQL database, allowing developers to perform database operations efficiently without writing complex SQL queries. Database connectivity is supported by MySQL Connector/J, ensuring reliable and secure data transactions. To maintain confidentiality of stored content, cryptographic libraries such as javax.crypto are imported to provide secure hashing algorithms like SHA-256 and encryption standards like AES-256 for protecting files at rest.

Within the machine learning microservice, Python imports several specialized libraries to support intelligent threat detection. Scikit-Learn supplies a robust collection of classification and anomaly detection algorithms that enable recognition of malicious files based on extracted features. Pandas plays a major role in preprocessing, cleaning, structuring, and managing file metadata and behavioral attributes before model prediction, while NumPy performs high-speed mathematical operations required for calculating entropy values, byte distribution patterns, and vector transformations. The microservice communicates with the backend through lightweight APIs built using Flask or FastAPI, enabling near real-time scanning without interrupting the user experience. These ML libraries together enhance the analytical intelligence of the system, allowing it to detect zero-day attacks and obfuscated malware more effectively than traditional signature-based solutions.

For transparency and accountability, the system integrates blockchain logging using libraries such as Web3j (or equivalent blockchain SDKs), which allow the backend to interact with a private blockchain network, create transactions, and store tamper-proof audit logs for every critical event including suspicious uploads, file access, and permission modifications. This ensures traceability and protects logs from manipulation or deletion by internal or external attackers. Finally, the frontend developed using React.js imports frameworks and helper packages such as React Router for secure page navigation and Axios for performing encrypted API calls to the backend. UI libraries such as Bootstrap or Material UI contribute to a responsive and visually intuitive dashboard for users and administrators, helping them interact efficiently with file management and alert monitoring features. Together, all these imported libraries form the structural and functional backbone of the system, enabling secure communication, intelligent automation, strong audit protection, and a smooth user experience across the entire application.

5.1.1 System working

The proposed system operates as a secure, intelligent, and accountable platform for storing and sharing user files while preventing malicious content from entering the environment. The workflow begins when a new user registers and logs in using secure authentication implemented through Spring Security. Once authenticated, the user gains access to core functionalities such as uploading files, downloading previously stored content, and managing file access permissions. All communications between the user interface and backend are encrypted through HTTPS to prevent interception or manipulation of data during transit.

When a user uploads a file, the system does not immediately store it. Instead, the Java backend initiates a multi-stage security validation process. The uploaded file is temporarily held in memory while essential metadata such as file size, file extension, MIME type, and cryptographic file hash are extracted. Additionally, the system analyzes content-specific values like entropy and structural characteristics that often reveal hidden malware or obfuscation patterns. A set of behavioral features such as user role, upload time pattern, and access history is also collected to determine the context of the action.

These extracted details are then securely transmitted to the Python-based Threat Detection Service, which operates as a standalone microservice. Using machine learning models built with Scikit-Learn and supported by Pandas and NumPy data processing, the Python module evaluates

the file and assigns a risk score. The score is compared against a defined threshold to classify the file as safe, suspicious, or malicious.

If the file is determined to be safe, the backend encrypts the content using strong cryptography (such as AES-256) and transfers it into the secure file storage repository. A corresponding metadata entry is recorded in the MySQL database, linking the file to its owner and access permissions. At the same time, a summary of the upload event (including file ID, user ID, and detection result) is stored immutably in the blockchain ledger to ensure that no file activity can be tampered with or erased.

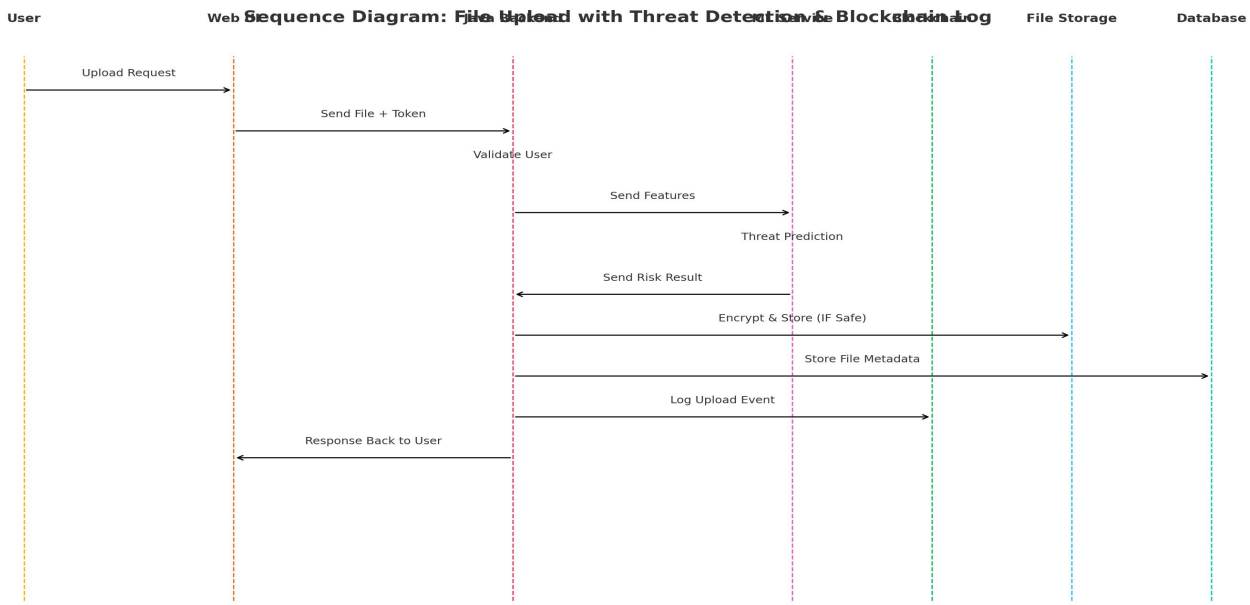

Figure 4 Sequence diagram for File Storage & Sharing System with Threat Detection and Blockchain - Based Audit

If the detection module classifies the upload as suspicious or malicious, the system immediately blocks the upload and prevents the file from being permanently stored. A security alert is generated and logged both in the database and blockchain, and the administrator is notified through the monitoring dashboard. This ensures malicious files are intercepted before they pose any threat to other users or system infrastructure.

Authorized users may later download their files through a secure request flow. When a download request is made, the backend verifies the user's access rights, decrypts the file on the server side, and delivers it over an encrypted communication channel. All downloads are logged for audit and compliance purposes. Meanwhile, administrators monitor system-level activities, threat alerts, blockchain audit logs, and user behavior trends to proactively detect abuse and

enforce policy compliance. Importantly, administrators cannot access the file contents, preserving user privacy and reducing insider risk.

5.1.2 Data Preprocessing

Data processing is a vital phase in the secure file storage and threat detection workflow, acting as the bridge between raw uploaded content and intelligent security decisions. When a user uploads a file, the system does not immediately place it into storage. Instead, the backend initiates a controlled data processing pipeline where the file undergoes validation, formatting, and transformation to prepare it for machine learning analysis. This ensures that every file is standardized into a structure the threat detection module can assess accurately and efficiently.

The processing begins with secure file acquisition, where the system temporarily receives the raw file data and validates its integrity, type, and upload permissions based on the user's role and authentication token. No data is trusted at this point. The system extracts technical metadata such as file size, file extension, header information, and timestamps, while simultaneously generating cryptographic identifiers like SHA-256 hash to uniquely represent the file. This prevents duplicate uploads and supports malware fingerprinting.

Next, the system proceeds to feature transformation, where static and behavioral attributes are converted into machine-interpretable formats. Continuous values like entropy, byte frequency, or upload frequency are normalized into consistent numeric scales. Meanwhile, categorical features such as file types or user roles are encoded into binary or vectorized forms to ensure compatibility with ML algorithms. Upload-context data such as IP origin, login history, and access pattern statistics are merged with the primary feature set, forming a comprehensive representation of both the file and the intent behind its upload. All this structured data is placed into a secure Pandas DataFrame format within the Python microservice for ML execution.

During this stage, data cleaning helps eliminate noise and inconsistencies. Invalid file formats, corrupted payloads, and incomplete uploads are filtered and logged. Any missing or malformed feature values are corrected, defaulted, or dropped depending on security rules. This cleanup ensures the ML detection model receives high-quality data while avoiding misclassification due to input errors.

The processed feature vector is then passed to the trained ML classification model, which calculates a risk probability score. The model compares extracted feature behavior with

previously learned malicious and safe patterns. If the risk score exceeds a defined threshold, the upload is immediately flagged as suspicious. In such cases, the system blocks permanent storage and triggers a blockchain-recorded alert for administrator monitoring. Conversely, if the file is assessed as safe, the backend encrypts the file and moves it into secure storage while storing its metadata and detection result in the database.

Finally, all processing actions are securely logged in both the database and the blockchain ledger. This establishes non-repudiation, ensures strong accountability, and supports post-incident forensic analysis. To maintain adaptability, periodic retraining and incremental learning can be applied as new real-world file patterns emerge, allowing the system to evolve against zero-day and polymorphic attack methods.

5.1.3 Feature Scaling and Transformation

Feature scaling and transformation form an essential stage in the threat detection pipeline of the proposed secure file storage and sharing system. After feature extraction, the system obtains a collection of heterogeneous attributes such as file size, entropy value, number of embedded objects, upload frequency, user role, and time-based behaviors. These features exist on very different numerical ranges and formats. For example, file size may be in megabytes, entropy in the range 0–8, and upload count may be a small integer, while some attributes are categorical strings such as file type (.pdf, .exe) or user role (User, Admin). If these raw values are directly fed into a machine learning model, larger-scale features can dominate the learning process, causing biased decision boundaries and poor generalization. To overcome this, the system applies systematic feature scaling and transformation to convert all attributes into a balanced, machine-interpretable format.

For numeric features, the system mainly uses two common scaling techniques: normalization and standardization. Normalization (often min–max scaling) rescales a feature to a fixed range, typically [0, 1]. If x is a value, x_min is the minimum, and x_max is the maximum, the normalized value is:

$$X' = X_{max} - X_{min} / X - X_{min}$$

This is particularly useful for features like file size, number of embedded objects, or upload counts, where absolute magnitudes differ widely. Standardization, on the other hand, transforms features to have zero mean and unit variance, using:

where μ is the mean and σ is the standard deviation. This is applied to features such as entropy or byte-level statistics, making them suitable for algorithms like SVM or Logistic Regression that assume approximately standardized data. By scaling numeric features, the system ensures that no single attribute (for example, a very large file size) arbitrarily dominates the model's decision.

In addition to scaling numeric values, the system performs categorical feature transformation. Attributes like file extension, MIME type, or user role are originally symbolic (e.g., "pdf", "exe", "image", "User", "Admin") and must be converted into numeric representations. The system uses techniques such as one-hot encoding, where each category is represented as a separate binary column. For instance, file type may become [is_pdf, is_exe, is_docx, …]. This prevents the model from incorrectly assuming any ordinal relationship between categories, while still allowing it to learn that executable files or certain rarely used formats tend to be more risky.

For some skewed or heavy-tailed numeric features, the system can apply non-linear transformations such as logarithmic scaling. File sizes or upload counts often vary by orders of magnitude; taking a log transform compresses the scale and reduces the impact of extreme outliers, making patterns easier for the model to learn. Similarly, counts of suspicious events or login attempts can be log-transformed before scaling to avoid overemphasizing rare but enormous values.

All feature scaling and transformation steps are implemented as part of a consistent preprocessing pipeline in the Python threat detection service. During training, the scaler (e.g., StandardScaler or MinMaxScaler from Scikit-Learn) learns the parameters (means, variances, min, max) from the training dataset and stores them. During real-time prediction, the same learned parameters are applied to incoming upload features to ensure that training and inference are perfectly aligned. This prevents data leakage and keeps the model's behavior stable over time.

5.1.4 Model Evaluation and Tuning

Model evaluation and tuning are critical phases in ensuring that the machine learning component of the proposed secure file storage and threat detection system performs accurately,

reliably, and consistently in identifying harmful file uploads. After developing the feature extraction and preprocessing pipeline, the model is trained using a labeled dataset consisting of both benign and malicious file samples. To ensure that the model does not simply memorize the training data (overfitting), the dataset is divided into training, validation, and testing portions, allowing proper assessment of the model's ability to generalize to unseen data.

During evaluation, multiple performance metrics are computed to provide a comprehensive understanding of the model's effectiveness. Accuracy gives an overall indication of correct predictions, but it is not sufficient for cybersecurity applications where detecting malicious files is more critical than correctly labeling normal ones. Therefore, metrics such as precision, recall, and F1-score are used. Precision ensures that files predicted as malicious are truly harmful, reducing false alarms, while recall ensures that actual threats are not missed by the system. The F1-score balances both, making it a key indicator for threat detection tasks. Additionally, the Confusion Matrix visually compares true positives, false positives, true negatives, and false negatives, helping administrators fine-tune classification thresholds. ROC-AUC (Receiver Operating Characteristic – Area Under Curve) further evaluates model discrimination ability, ensuring that safe and malicious patterns remain distinct even under varied conditions.

To improve model performance, hyperparameter tuning is performed. Machine learning algorithms contain adjustable settings (e.g., number of trees in Random Forest, kernel selection in SVM, contamination rate in Isolation Forest) that greatly influence model quality. Techniques such as Grid Search and Random Search are used to systematically test various parameter combinations using the validation set, identifying the optimal configuration that yields the highest detection performance. In certain cases, Bayesian Optimization can also be applied for faster convergence toward best-performing settings.

The system also uses cross-validation, especially k-fold cross-validation, where the dataset is repeatedly split into multiple train-test subsets. This prevents bias due to a single random split and ensures consistent results across different data distributions. Cross-validation is extremely important when dealing with imbalanced datasets, where malicious files are fewer compared to benign ones. To address imbalance, techniques like SMOTE (Synthetic Minority Oversampling Technique) or class-weight adjustments may be applied so the model does not learn to classify most uploads as safe.

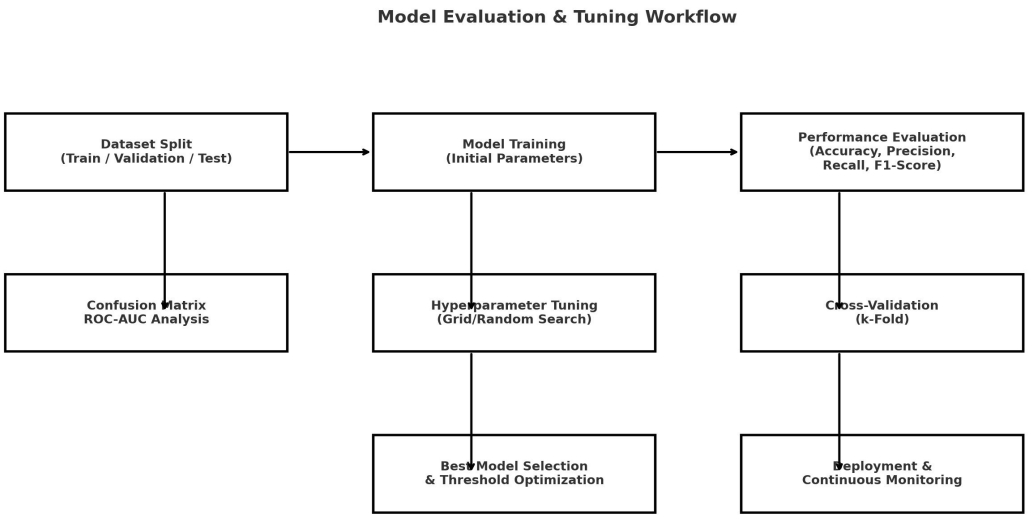

Figure 5 Model Evalution

5.1.5 User Interface Design

The user interface of the proposed Secure File Storage and Threat Detection System is designed to be clean, intuitive, and security-oriented, allowing both normal users and administrators to interact with the system efficiently without being exposed to unnecessary complexity. The application follows a web-based UI model developed using React.js, structured into clearly defined screens such as Login, User Dashboard, File Management, and Admin Dashboard. Consistent layout, typography, and color themes are maintained across all pages to provide a professional and cohesive experience. Navigation is primarily handled through a top or side menu bar, giving quick access to the major features while ensuring that restricted options are visible only to authorized roles.

The Login Screen serves as the entry point to the system and emphasizes security and simplicity. It contains fields for username and password, along with error messages for invalid credentials or blocked accounts. Password input is masked and all communication is performed over HTTPS, reinforcing the secure nature of the system. A subtle indication of system status (e.g., “Secure File Storage Portal”) and optional links to help or contact support may be included. Once authenticated, users are redirected to different dashboards depending on their role: User or Admin.

The User Dashboard is designed to provide a clear overview of that user’s own activity and file status. It typically contains panels or sections such as “My Files”, “Recent Uploads”, and

“Security Notifications”. From this dashboard, a user can access the File Upload Interface, which includes a file picker, drag-and-drop area, and selectable options like description or tags. A progress bar and status messages inform the user while the file is being uploaded and scanned. After the threat detection step completes, the UI displays whether the upload was successful or blocked, along with a concise message such as “File stored securely” or “Upload blocked due to security risk”. At no point is the detection logic exposed, but the user is aware that security checks are being performed.

The File Management Screen presents a table or card-based list of all files owned by the logged-in user. Each entry typically displays filename, size, upload date, status (safe/suspicious/blocked), and available actions such as Download or (optionally) Share. For security reasons, previews of file content are not automatically shown; instead, the user intentionally downloads the file if needed. Access control rules are enforced behind the scenes, so only allowed actions are enabled for each row. Clear icons and tooltips are used to indicate whether a file passed the threat scan and whether any warnings were associated with it.

The Admin Dashboard is visually distinct but follows the same design language. It focuses on system-wide monitoring rather than file content access. Typical sections include “Threat Alerts”, “Suspicious Upload Attempts”, “User Activity Overview”, and “Blockchain Audit Log Summary”. Admins can view high-level metadata such as which user attempted to upload a suspicious file, when it occurred, and how it was classified, but they cannot open or view the actual file content, maintaining user privacy. Filter and sorting controls allow the admin to quickly investigate risky accounts, repeated failed logins, or abnormal activity patterns. A separate User Management view allows admins to change roles, block or unblock accounts, and review account status, all via safely designed forms and confirmation dialogs to avoid accidental misuse.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

The proposed Secure File Storage and Threat Detection System successfully integrates modern security technologies to provide a protected and trustworthy platform for handling sensitive digital files. By combining encryption, machine learning-based malware analysis, access control enforcement, and blockchain-enabled audit logging, the system ensures confidentiality, integrity, and traceability throughout all file transactions. Unlike traditional cloud file storage services that focus mainly on basic sharing capabilities, this solution proactively prevents malicious content from entering the environment by analyzing uploaded files in real-time, blocking threats before they can propagate or cause harm.

The modular architecture implemented using Java Spring Boot and a Python microservice ensures scalability and maintainability, allowing continuous enhancement of detection techniques as cyberthreats evolve. The integration of blockchain removes any possibility of tampering with audit trails, thereby strengthening accountability and supporting forensic analysis during security incidents. At the same time, the user-friendly web interface offers easy file management for users while ensuring that privacy remains uncompromised, even from administrators with elevated privileges.

Overall, this system demonstrates a forward-thinking approach to secure cloud data handling, showing how intelligent cybersecurity mechanisms can be seamlessly integrated into everyday file operations. It not only addresses current vulnerabilities such as ransomware-infected uploads, unauthorized access, and insider threat manipulation but also lays a robust foundation for future research and enhancements, including deeper behavioral analytics, advanced anomaly detection, and large-scale enterprise deployment. This project stands as a complete and efficient solution for organizations demanding both strong data protection and real-time threat resilience.

6.2 FUTURE ENHANCEMENT

Although the proposed Secure File Storage and Threat Detection System effectively prevents malicious uploads and ensures trust through blockchain auditing, there are several opportunities for enhancement to improve its intelligence, usability, and adaptability to evolving cyber risks. One major enhancement involves upgrading the machine learning component to incorporate deep learning models such as Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) capable of analyzing raw binary data and file behavior patterns in more depth. This would help improve the detection rate of zero-day threats and polymorphic malware that evades static feature-based analysis. Furthermore, integrating continuous model retraining using newly flagged suspicious files will allow the system to become more adaptive, powering automated learning over time as threat landscapes change.

Another promising improvement is adding sandbox-based dynamic analysis. In this approach, the system would temporarily execute the uploaded file in a secure isolated environment and observe its behavior before making a decision. Dynamic analysis would provide additional threat indicators such as unauthorized file modifications, registry operations, or network communication attempts, leading to more accurate detection of hidden or dormant malware payloads. A hybrid model that combines both static and dynamic analysis can significantly reduce false positives and offer higher confidence in detection outcomes.

On the privacy and security front, the system can be enhanced to support end-to-end encryption between shared users, integrating public-key cryptography so that only the intended recipient can decrypt shared files. Role-based visibility rules can be expanded to include fine-grained access policies such as time-restricted access, watermarking for file traceability, or attribute-based access control (ABAC) to further protect sensitive content. In addition, a file versioning and recovery feature may be incorporated to counter ransomware-style corruption attempts by automatically backing up previous versions of files.

From a blockchain perspective, the system can evolve from a simple private blockchain to a permissioned distributed ledger network with multiple nodes hosted across organizational environments. This will further decentralize audit data and ensure even stronger resistance against insider attacks or log tampering. Smart contract integration can also automate policy enforcement, such as auto-blocking malicious accounts flagged repeatedly or requiring administrator approval for high-risk uploads.

REFERENCES

1. **J. Jayachitra, H. Arshad and R. Muthukumaran, "Cloud Storage and Secure File Sharing Using Blockchain with Xchacha20 and Shake Cryptography," 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN),**

2. **Songling FU, Xiangke LIAO, Lianyue HE, Chenlin HUANG, Xiaodong TANG, Si ZHENG. Efficient and Fine-Grained Sharing of Encrypted Files .Institute of Electrical and Electronics.**

3. **"Blockchain for Secure Storage Systems" – IEEE Xplore, 2020.**

4. **Behrouz A. Forouzan – Cryptography and Network Security (McGraw-Hill, 2015).**

5. **OWASP Foundation – OWASP Top 10 Security Risks (2021)..**

6. **Madhu Sharma, Dr. Ashish Sharma, A secret file sharing scheme with chaos-based**

7. **encryption. Institute of Electrical and Electronics Engineers.2019.**

8. **[5] NIST (National Institute of Standards and Technology) – Guidelines for Key**

9 . Management (NIST Special Publication 800-57).Bhattacharjee, S., Mukherjee, S. *Cybersecurity: Cyber Attacks, Protection, and Future Trends*, Springer, 2021.

10 . Scarfone, K., and Mell, P. *Guide to Intrusion Detection and Prevention Systems (IDPS)*, National Institute of Standards and Technology (NIST) Special Publication 800-94, U.S. Department of Commerce, 2022.

. **11. Nakamoto S. Bitcoin: A peer-to-peer electronic cash system. Decentralized Bus. Rev. 2017:21260.**

APPENDICES

UI Folder Structure:

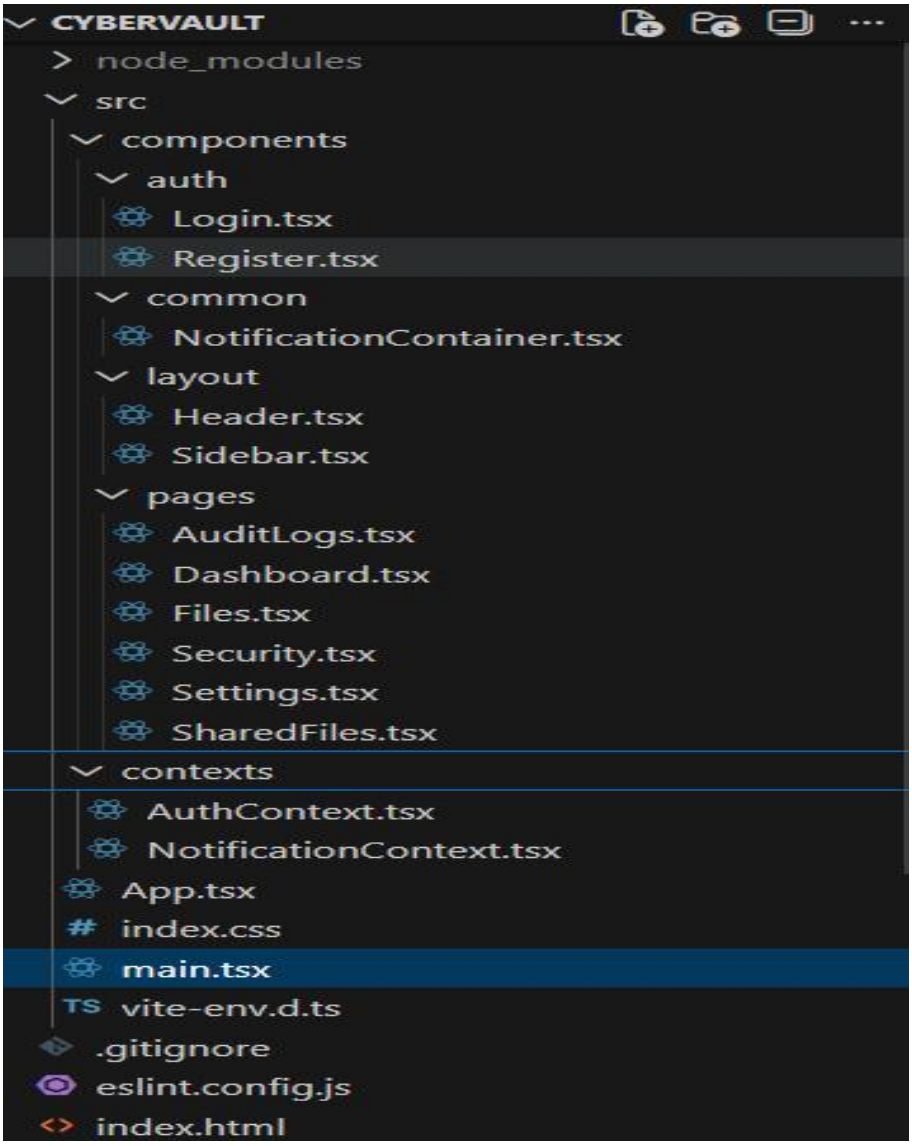

Figure 9 UI Folder structure for File Storage and Sharing System

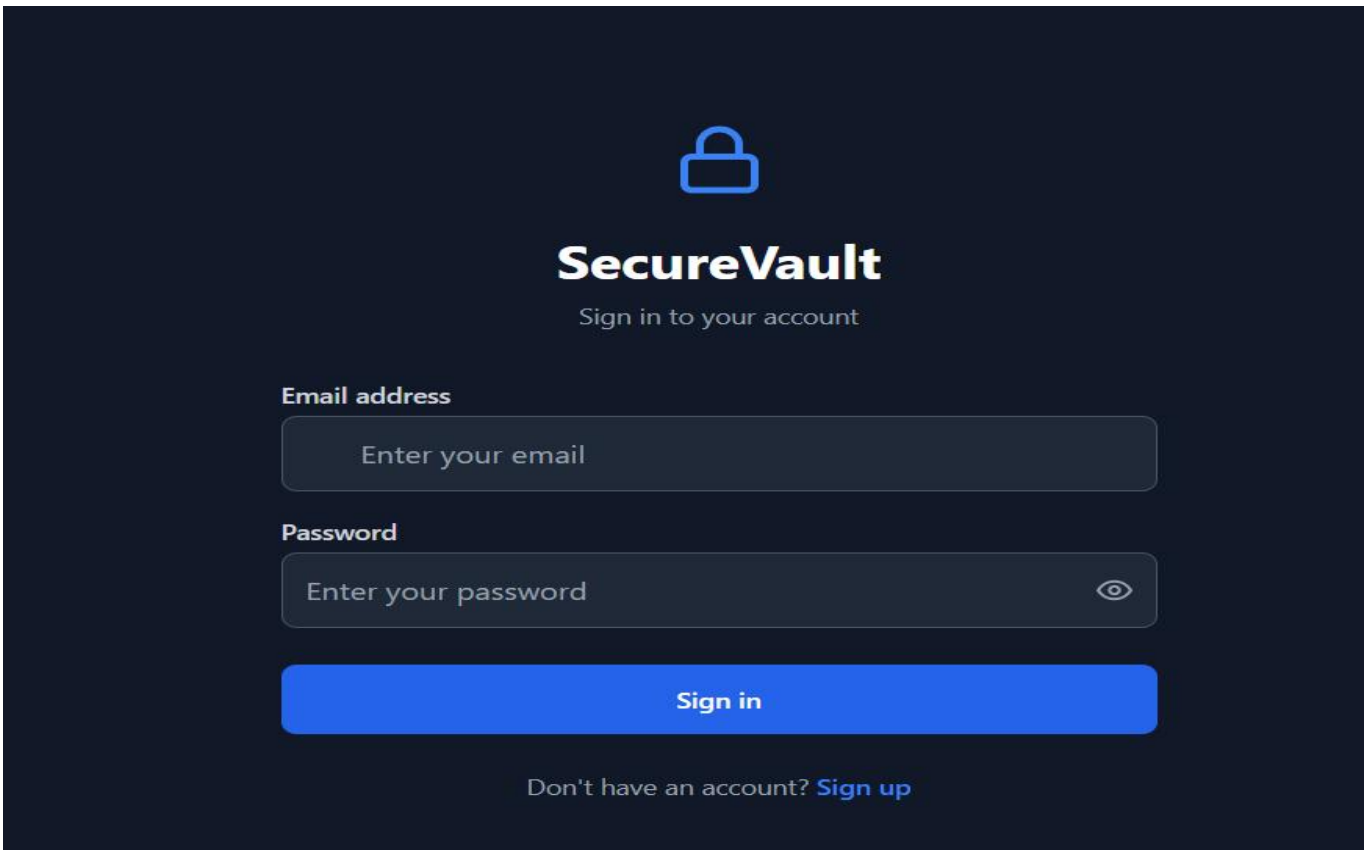

Figure 9 UI for File Storage and Sharing System

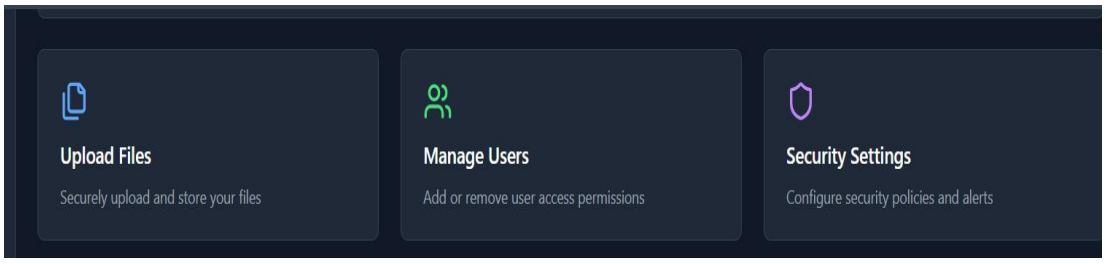

Figure 9 Options in File Storage and Sharing System

Source code for UI:

React Frontend Development

App.jsx

```
import React, { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Sidebar from './components/layout/Sidebar';
import Header from './components/layout/Header';
import Dashboard from './components/pages/Dashboard';
import Files from './components/pages/Files';
import SharedFiles from './components/pages/SharedFiles';
import Security from './components/pages/Security';
import AuditLogs from './components/pages/AuditLogs';
import Settings from './components/pages/Settings';
import Login from './components/auth/Login';
import Register from './components/auth/Register';
import { AuthProvider, useAuth } from './contexts/AuthContext';
import { NotificationProvider } from './contexts/NotificationContext';
import NotificationContainer from './components/common/NotificationContainer';

function AppContent() {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return (
      <div className="min-h-screen bg-gray-900 flex items-center justify-center">
        <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-500"></div>
      </div>
    );
  }

  if (!isAuthenticated) {
    return (
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="*" element={<Navigate to="/login" replace />} />
      </Routes>
    );
  }

  return (
    <div className="min-h-screen bg-gray-900 flex">
      <Sidebar />
      <div className="flex-1 flex flex-col">
        <Header />
        <main className="flex-1 p-6 overflow-auto">
          <Routes>
            <Route path="/dashboard" element={<Dashboard />} />
            <Route path="/files" element={<Files />} />
            <Route path="/shared" element={<SharedFiles />} />
            <Route path="/security" element={<Security />} />
            <Route path="/audit" element={<AuditLogs />} />
            <Route path="/settings" element={<Settings />} />
            <Route path="/" element={<Navigate to="/dashboard" replace />} />
          </Routes>
        </main>
      </div>
      <NotificationContainer />
    </div>
  );
```

```
}

function App() {
  return (
    <Router>
      <AuthProvider>
        <NotificationProvider>
          <AppContent />
        </NotificationProvider>
      </AuthProvider>
    </Router>
  );
}

export default App;
```

Backend

User.java

```
package com.example.demo.model;
import jakarta.persistence.*;
import lombok.*;
@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password; // stored as bcrypt hash

    private String role; // e.g. "ROLE_USER"
}
```

StoredFile.java

```
package com.example.demo.model;
```

```
import jakarta.persistence.*;
import lombok.*;
@Entity
@Table(name = "files")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class StoredFile {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String originalFileName;
    private String storedFileName;
    private String filePath;
    @Lob
    private String encryptedAesKey;    // AES key encrypted with RSA
    @Lob
    private String ivBase64;           // IV for AES
    private long size;
    //      NEW PART: link file to User (owner)
    @ManyToOne
    @JoinColumn(name = "owner_id")      // column name in DB
    private User owner;
}
```